Declarative Provisioning of Synrc VE with Terraform and Packer

Synrc Hypervision Research

August 25, 2026

Abstract

This paper presents a fully declarative architecture for provisioning and managing the Synrc Virtualization Environment (Synrc VE / OS.1) using the HashiCorp stack. Building upon the seL4-based Synrc Hypervision [1], we show how Packer produces reproducible golden images for both conventional guests and high-assurance unikernels, while Terraform orchestrates their lifecycle on Proxmox VE and integrates with Kubernetes-compatible control planes. The approach yields a single source of truth in Git, strong auditability, and minimal trusted computing base suitable for high-assurance and public-sector deployments.

Contents

1 Introduction (Functional Declaration)

Modern infrastructure demands both strong isolation guarantees and operational simplicity. The Synrc Hypervision project [1] addresses the former by hosting an unmodified Erlang/OTP BEAM runtime inside a formally verified seL4 Microkit protection domain, while confining device drivers to a minimal Alpine Linux guest. Communication occurs exclusively through capability-mediated shared-memory channels (sDDF / VirtIO).

The resulting system dramatically reduces the Trusted Computing Base (TCB) compared with conventional Linux-based hypervisors, yet retains practical hardware support. Complementary documents describe Proxmox VE integration [2], a high-assurance orchestration plane (synrc-kube) [3], VirtIO architecture [4], and the Chimera-based control plane [5].

Operationalising this architecture at scale, however, requires a declarative, reproducible provisioning layer. Manual creation of LXC containers and QEMU/KVM virtual machines on Proxmox VE, or ad-hoc bootstrap of Kubernetes nodes, quickly becomes error-prone and unauditable.

We therefore declare the following functional requirements:

1. All infrastructure state shall be expressed in HashiCorp Configuration Language (HCL).
2. Golden images for operating-system guests and unikernels shall be built reproducibly by Packer.
3. Terraform shall be the sole tool that creates, updates and destroys resources on Proxmox VE.
4. Secrets (Proxmox API tokens, SSH keys, kubeconfig material) shall reside in HashiCorp Vault.
5. The entire definition shall live in a single Git repository, enabling review, audit and deterministic recreation of any environment.

The resulting repository layout is:

```
+--- Packer (image construction)
|   +-- base-os.pkr.hcl
|   +-- k8s-node.pkr.hcl
|   +-- guest-netbsd.pkr.hcl
|   +-- guest-alpine.pkr.hcl
|   +-- guest-freebsd.pkr.hcl
|   +-- unikernel-beam.pkr.hcl
|   +-- unikernel-jvm.pkr.hcl
|   +-- unikernel-clr.pkr.hcl
|   +-- cow-node.pkr.hcl          # Proxmox qcow2 templates
|   +-- podman-host.pkr.hcl
+--  Terraform
|   +-- modules/proxmox/          # VMs, LXC, networks, storage
|   +-- modules/kubernetes/       # node provisioning & bootstrap
|   +-- environments/             # dev / stage / prod
+-- Vault                         # secrets, tokens, kubeconfig
```

This paper describes how the above declaration is realised.

2 Terraform for Synrc VE

Terraform acts as the declarative orchestrator of the Synrc Virtualization Environment. It never builds images; it only consumes Packer-produced artefacts that have been registered as Proxmox templates or LXC ostemplates.

2.1 Kubernetes and Proxmox VE Support

Proxmox VE is the primary target hypervisor for AMD64 deployments (VirtIO / KVM). Terraform uses the `bpg/proxmox` provider to manage:

- Virtual machines that host seL4 + Synrc BEAM unikernels,
- LXC containers that run Alpine-based Erlang micro-services (LDAP, CA, KVS, VPN, IAS) as described in PROXMOX.md,
- Conventional Kubernetes worker and control-plane nodes (when a hybrid cluster is required),
- Software-defined networks, firewall rules and storage volumes.

Kubernetes support is realised in two complementary ways:

1. **Classical path** — Terraform provisions Ubuntu/Alpine VMs from the `k8s-node` Packer image; a subsequent provisioner or external GitOps controller bootstraps kubeadm / RKE2 / k3s with CRI-O or containerd. Rootless Podman is available on the same nodes for system services and `podman kube play` compatibility.

2. **High-assurance path (synrc-kube)** — Terraform instantiates seL4-based unikernel VMs. The control-plane services (KVS replacing etcd, Name Service, Certificate Authority, Identity) run as isolated BEAM protection domains. OCI images are reinterpreted as seL4 guest descriptors rather than Linux containers [3].

Both paths share the same Terraform modules; only the chosen Packer template identifier changes.

2.2 Supported Operating Systems

The Synrc bootloader and system descriptions support a range of guests that can be selected at deployment time:

Guest	Role
Alpine Linux (musl)	Primary driver domain and LXC micro-services
NetBSD	Alternative System-plane flavour, rump-kernel guests
FreeBSD	Full BSD userland guest / rump compatibility
Chimera Linux	Preferred control-plane OS (musl + FreeBSD userland)
Ubuntu / Debian	Conventional Kubernetes nodes (when required)

Additional embedded targets (NuttX, LK, BTRON) are recognised by the bootloader but are outside the scope of the present Terraform modules.

2.3 Supported Unikernels (BEAM, JVM, CLR)

BEAM (Erlang/OTP) The primary and fully implemented unikernel. An unmodified Ericsson BEAM is linked against the Synrc syscall provider (≈ 1 kLOC) and executes as an seL4 protection domain. Device I/O is performed via VirtIO front-ends backed by a capability-isolated host domain that owns the physical NVMe controller [4].

JVM Target unikernel. Packer produces a minimal image containing a statically linked GraalVM native-image or a stripped OpenJDK runtime together with a thin Synrc-compatible syscall layer. Lifecycle management is identical to BEAM instances.

CLR (.NET) Target unikernel. Analogous packaging of a CoreCLR / NativeAOT runtime. Both JVM and CLR images are registered as Proxmox templates and instantiated by the same Terraform resources used for BEAM.

All unikernels share the same VirtIO transport and capability model; only the application runtime binary differs.

2.4 Declarations (OS and Unikernels)

2.4.1 BEAM Unikernel VM

A typical Terraform declaration for a high-assurance BEAM unikernel on Proxmox VE looks as follows:

```
resource "proxmox_virtual_environment_vm" "beam_unikernel" {
  name      = "hv-beam-01"
  node_name = "pve01"
  vm_id     = 9100
  clone {
    vm_id = var.unikernel_beam_template_id   # produced by Packer
    full  = true
  }
  cpu {
    cores = 2
    type  = "host"
  }
  memory {
    dedicated = 4096
  }
  disk {
    datastore_id = "local-lvm"
    interface    = "virtio0"
    size         = 20
    file_format  = "qcow2"
  }
  network_device {
    bridge = "vmbr0"
    model  = "virtio"
  }
  initialization {
    user_account {
      username = "synrc"
      keys     = [file(var.ssh_public_key)]
    }
  }
}
```

2.4.2 NetBSD MicroVM

For lightweight, low-footprint System-plane services or rump-kernel guests, a NetBSD MicroVM is declared with minimal CPU and memory allocations:

```
resource "proxmox_virtual_environment_vm" "netbsd_microvm" {
  name      = "hv-netbsd-microvm-01"
  node_name = "pve01"
  vm_id     = 9200
  clone {
    vm_id = var.netbsd_microvm_template_id
    full  = true
  }
  cpu {
    cores = 1
    type  = "host"
  }
  memory {
    dedicated = 512
  }
  disk {
    datastore_id = "local-lvm"
    interface    = "virtio0"
    size         = 4
    file_format  = "qcow2"
  }
  network_device {
    bridge = "vmbr0"
    model  = "virtio"
  }
}
```

2.4.3 Full NetBSD Guest VM

When a full NetBSD userland guest or extended POSIX environment is required:

```
resource "proxmox_virtual_environment_vm" "netbsd_full" {
  name      = "hv-netbsd-full-01"
  node_name = "pve01"
  vm_id     = 9210
  clone {
    vm_id = var.netbsd_full_template_id
    full  = true
  }
  cpu {
    cores = 2
    type  = "host"
  }
  memory {
    dedicated = 2048
  }
  disk {
    datastore_id = "local-lvm"
    interface    = "virtio0"
    size         = 20
    file_format  = "qcow2"
  }
  network_device {
    bridge = "vmbr0"
    model  = "virtio"
  }
}
```

2.4.4 Alpine Linux LXC Micro-service

LXC-based Erlang micro-services (such as LDAP, CA, VPN, KVS) are declared using an Alpine ostemplate built by Packer:

```
resource "proxmox_virtual_environment_container" "alpine_lxc" {
  node_name    = "pve01"
  vm_id        = 9300
  unprivileged = true

  initialization {
    hostname = "synrc-ldap"
    ip_config {
      ipv4 {
        address = "192.168.1.101/24"
        gateway = "192.168.1.1"
      }
    }
    user_account {
      keys = [file(var.ssh_public_key)]
    }
  }

  operating_system {
    template_file_id = "local:vztmpl/synrc-ldap-alpine.tar.zst"
    type             = "alpine"
  }

  cpu {
    cores = 1
  }
  memory {
    dedicated = 512
  }
  disk {
    datastore_id = "local-lvm"
    size         = 8
  }
  network_interface {
    name   = "eth0"
    bridge = "vmbr0"
  }
}
```

Environment-specific values (template IDs, network CIDRs, resource sizes) are supplied via `tfvars` files under `environments/`, keeping the modules pure and reusable.

3 Conclusion

By treating Packer as the sole image factory and Terraform as the sole infrastructure orchestrator, the Synrc Virtualization Environment obtains a completely declarative, Git-centric lifecycle. The same HCL definitions can instantiate:

- classical Kubernetes clusters on Proxmox VE with rootless Podman,
- high-assurance seL4 + BEAM (and future JVM/CLR) unikernels,
- supporting Alpine/NetBSD/FreeBSD guests and Erlang micro-services.

The approach preserves the formal isolation properties of seL4 while giving operators the familiar plan/apply workflow, audit trail and multi-environment consistency required in regulated and public-sector settings. Future work includes native Terraform providers for synrc-kube control-plane objects and tighter Vault integration for capability tokens.

References

[1] N. Tonpa, *Synrc Hypervision*, https://github.com/synrc/hv, 2026.

[2] N. Tonpa, *Unikernel Deployments on Proxmox VE*, PROXMOX.md, 2026.

[3] N. Tonpa, *Kubernetes Deployments / synrc-kube*, KUBE.md, 2026.

[4] N. Tonpa, *VirtIO Architecture*, VIRTIO.md, 2026.

[5] N. Tonpa, *Control Plane (Chimera)*, CHIMERA.md, 2026.